PROJECT AND TEAM INFORMATION

Project Title

AI-Powered Semantic Search Engine for Intelligent File Retrieval

Student / Team Information

Team Name: *Team #*	*Testudo*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Chahal, Harsh, 240211874* *harshchahal7889@gmail.com* 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

The main motivation of this project is to solve a problem present in every file system today i.e.files can only be found by their exact name, not by what they actually contain. A normal Operating System treats a file as raw bytes with a label, it has no understanding of the information stored inside it. This becomes a real issue when a user remembers the content of a document but has forgotten its filename or location where it is stored, forcing them to open and manually search through several files.
Our project, a AI-Powered Semantic Search Engine for Intelligent File Retrieval, solves this by adding an intelligent search layer on top of a simulated file system. Instead of matching filenames or exact keywords, the system understands the meaning of a users query using Sentence-Transformer Model in other words all-miniLM-L6-v2 model and retrieves the most relevant file even if none of the search words appear in it directly. For example, a query such as "where did we discuss GPU purchases?" should correctly retrieve a file that only mentions "buying hardware".
This project is important because it combines three core subjects into one practical system: Operating Systems (simulated disk storage, block allocation, disk scheduling, and file permissions), Database Management Systems (structured storage of file metadata and vector embeddings), and Artificial Intelligence / NLP (sentence embeddings and similarity search). It shows a real-world problem solved by modern tools such as Google Drive and Notion, and helps us understand how low-level file storage and high-level language understanding can work together in a single, connected application.

State of the Art / Current solution (1 pt)

Currently, most operating systems and file explorers rely purely on filename or path-based search, and at best a basic keyword match against file content. This approach fails whenever a user does not remember the exact words used inside a document.
A more advanced approach used in industry is keyword-based statistical retrieval, such as TF-IDF (Term Frequency-Inverse Document Frequency), which ranks documents based on how frequently specific words appear. While better than plain filename search, this method still cannot understand synonyms or context searching for car will not match a document containing automobile.
The current state of the art solution, used by large-scale products like Google Drive, Notion, and enterprise search tools, is Semantic Search using Sentence Embedding and Vector Databases. Here, a deep learning model converts text into numerical vectors that capture meaning, and similarity search (cosine similarity) is used to retrieve conceptually related documents, regardless of exact wording.
My project implements a simplified academic version of this modern approach: a C++-based simulated OS-level file storage layer, a Python sentence-transformer model for Embeddings, and a relational database to store metadata and vectors together, allowing us to understand the complete flow before working with production scale distributed vector databases.

Project Goals and Milestones (2 pts)

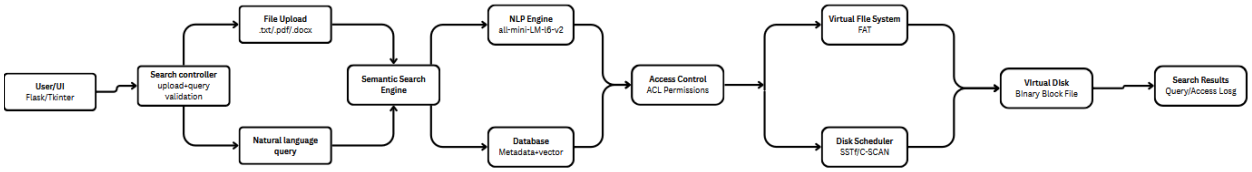
Project Goals:
1. Simulate a basic virtual file system with directories and disk blocks.
2. Implement a simulated disk allocation method (FAT) for storing files.
3. Implement a disk scheduling algorithm for simulated block retrieval.
4. Extract text content from uploaded files (.txt, .pdf, .docx etc).
5. Use an NLP sentence-transformer model to convert file text into vector Embeddings.
6. Design a relational database schema to store file metadata and Embeddings.
7. Implement semantic search using cosine similarity between query and file vectors.
8. Implement basic file permissions (ACL) to restrict illegal access to search results.
9. Build a simple user interface for uploading files and searching by natural language.
10. Log user queries and retrieved files for testing and evaluation.
11. Test search accuracy against traditional keyword-based search.
12. Analyze and document performance, accuracy, and limitations.

Milestones:
Milestone 1: *Study file system concepts, disk scheduling, and NLP embedding models.*
Milestone 2: *Implement the virtual file system and simulated disk storage.*
Milestone 3: *Implement disk scheduling for simulated block access.*
Milestone 4: *Design and create the database schema (metadata + embeddings).*
Milestone 5: *Integrate a sentence-transformer model for text-to-vector conversion.*
Milestone 6: *Implement the indexing workflow (file upload → text extraction → embedding → DB storage).*
Milestone 7: *Implement the semantic search workflow (query → embedding → similarity search).*
Milestone 8: *Add file permission checks before returning search results.*
Milestone 9: *Build the user interface and connect it to the backend.*
Milestone 10: *Perform testing, debugging, performance analysis, and documentation.*

Project Approach (3 pts)

I built this system to bridge core concepts across Operating Systems, Databases, and AI into a single semantic retrieval engine. Rather than relying on rigid keyword matching, the application allows users to search documents based on their actual meaning using a simple interface written in Flask or Tkinter. When a user uploads a document, we extract its raw text and feed it through the pretrained all-MiniLM-L6-v2 Sentence Transformer. This converts the text into a dense vector embedding that captures its core semantic meaning. We apply the exact same embedding pipeline to incoming user queries. To rank search results, the engine measures the Cosine Similarity between the query vector and pre-computed document embeddings. On the backend, data storage is split into two distinct operational layers:
Database Management: An SQLite or PostgreSQL database handles document metadata, vector embeddings, user account ownership details, and access control policies.
Virtual File System (OS Layer): We implemented a custom virtual file system in C++ to manage low-level disk operations. Text storage relies on fixed-size disk blocks organized via FAT-style allocation. Physical block access is simulated using disk scheduling algorithms like SSTF or C-SCAN. Before serving any file back to the end user, an Access Control List (ACL) validates the requesting user's explicit permissions to prevent unauthorized access.

System Architecture (High Level Diagram)(2 pts)



Project Outcome / Deliverables (1 pts)

The final project will deliver a functional Semantic Search Engine capable of retrieving files based on the meaning of a natural language query rather than exact filenames or keywords.

The major deliverables will include:

1. *A working virtual file system simulating disk blocks and file allocation.*
2. *A disk scheduling module for simulated block retrieval.*
3. *An NLP-based indexing workflow that converts file text into vector embeddings.*
4. *A relational database storing file metadata and embeddings.*
5. *A semantic search module using cosine similarity for query matching.*
6. *Basic file permission (ACL) checking on search results.*
7. *A functional user interface for uploading files and performing searches.*
8. *Query and access logs for evaluation.*
9. *Performance comparison results against traditional keyword search.*
10. *Source code and database scripts.*
11. *Project documentation and final presentation.*

The project will demonstrate the practical integration of Operating Systems, Database Management Systems, and Natural Language Processing / AI concepts within a single working application.

Assumptions

The project assumes that the file system and search engine will run in a controlled local environment on a single machine rather than a distributed setup. Files used for testing will primarily be text-based (.txt, .pdf, .docx) of moderate size. The sentence-transformer model used will run locally without requiring GPU acceleration. The database and virtual disk are assumed to be available locally and accessible to the application. User authentication is simplified, with basic role-based permissions rather than a full-scale security system.

References

Sentence Embeddings / Sentence-Transformers:
https://www.sbert.net/

Cosine Similarity:
https://en.wikipedia.org/wiki/Cosine_similarity

Disk Scheduling Algorithms (SSTF, C-SCAN):
https://www.geeksforgeeks.org/disk-scheduling-algorithms/

File Allocation Table / i-node Concepts:
https://www.geeksforgeeks.org/file-allocation-methods/

Vector Databases (FAISS / ChromaDB):
https://faiss.ai/
https://www.trychroma.com/